

DocProcessor

Turn documentation into a verifiable feature map for QA automation.

Summary

DocProcessor is a standalone, fully decoupled Go module that loads project documentation, builds structured feature maps, and tracks verification coverage. It is designed to work with LLM agents for intelligent feature extraction but also includes heuristic extraction for offline use.

Short description

A project-agnostic Go module for documentation processing and feature-map extraction. It parses docs into structured feature maps and tracks which features are verified — using LLM agents for intelligent extraction or heuristics offline — feeding QA automation with an anti-bluff, always-matches-reality guarantee.

Long description

Every software team lives with the same slow lie: the documentation promises features, the tests cover something adjacent, and nobody can say with confidence whether the two describe the same product. DocProcessor exists to make that gap visible and measurable. Given a project's documentation, it builds a structured feature map — an enumerated, machine-readable model of everything the product claims to do — and tracks verification coverage against it, so “is this documented feature actually proven?” stops being a hallway argument and becomes a query with an answer. It is deliberately dual-mode: it uses LLM agents for intelligent, semantic feature extraction when one is available, and falls back to a heuristic parser for fully offline use, so it never hard-depends on a model being present and runs identically in an air-gapped CI job or on a developer's plane-mode laptop.

Architecturally it is a standalone, project-not-aware, fully decoupled Go module (CONST-051(B)): it ships zero project-specific values and is incorporated by consumers as an equal-codebase submodule, so any project can adopt it without inheriting someone else's assumptions. It also holds itself to the standard it enforces on others — its own claims are bound by the anti-bluff covenant (CONST-035) and full-automation coverage rules (CONST-048), meaning every capability its README advertises is exercised by an automated test or Challenge script that confirms real, end-user-usable behavior rather than merely exiting zero; user-facing strings route through the CONST-046 i18n translator seam. The point of all this is a closed loop: DocProcessor is the input side of the QA cycle that HelixQA closes — it extracts the feature map from the docs, HelixQA proves each mapped feature with captured runtime evidence, and documentation, tests, and shipped behavior are forced to converge instead of quietly drifting apart release after release.

Why we built it

Documentation and tests drift apart: docs promise features that no test proves, and QA can't easily tell what "complete" means. DocProcessor turns docs into a machine-readable feature map so verification coverage can be measured against what was actually promised.

Why it's a game-changer

It converts the fuzziest question in software delivery — "does what we shipped match what we said we shipped?" — into something automatable and continuously checkable, and it does so without a hard AI dependency: LLM extraction when a model is available, heuristics when it isn't, so the same guarantee holds in every environment from an offline runner to a fully-agentic pipeline.

What's innovative

- Documentation-to-feature-map extraction with verification-coverage tracking.
- Dual extraction: LLM-agent-driven or heuristic/offline.
- Project-agnostic, zero-config decoupling (CONST-051(B)).
- Anti-bluff self-verification: README claims are backed by tests/Challenges (CONST-035/048).

Challenges & solutions

- **Model-optional operation:** solved with a heuristic extractor fallback so the module works offline.
- **Keeping docs and reality aligned:** solved by structured feature maps + verification coverage tracking, tied into the QA loop.
- **Reusability:** solved via strict decoupling and equal-codebase submodule consumption.
- **Credibility of its own claims:** solved with anti-bluff tests/
Challenges for every advertised capability.

Tech stack (why + how)

- **Go (1.25+)** — module core; Apache-2.0 licensed.
- **LLM agents** — intelligent semantic feature extraction (optional).
- **Heuristic parser** — offline feature extraction fallback.
- **i18n Translator (pkg/i18n)** — CONST-046 localized strings.
- **Challenge harness** — anti-bluff verification of the module's own claims.